

Annexure A – Analytical Problem Solving

1. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list. (a) L1.append (10); L1.append (20); L1.append (15); (b) L2.append (10); L2.append (20); L2.append (15); L2.moveToStart (); L2.insert (39); L2.next (); L2.insert (12). (20 Marks)
2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail. (20 Marks)
3. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77). Pop(), Push(44), Push(33), Pop(), Pop(),Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. (20 Marks)
4. Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to the following expressions
 - a. Prefix expression
 - b. Postfix expression.(20 Marks)
5. A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors, partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge	Course Coordinator	Program Director			

Name : S. Jaxon Rizal

Reg. No : 192511421

Analytical Problem Solving (02-ATI)

1. Linked List (List ADT)

(a) Operations:

L1.append(10)

L1.append(20)

L1.append(15)

Result:

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 15 $\rightarrow$ NULL

Current Position (curr) : At 15 (last node)

(b) Operations:

L2.append(10)

L2.append(20)

L2.append(15)

L2.moveToStart()

L2.insert(39)

L2.next()

L2.insert(12)

Step-by-Step:

1.

10 $\rightarrow$ 20 $\rightarrow$ 15

2. move To Start ()

curr $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 15

3. insert (39)

10 $\rightarrow$ 39 $\rightarrow$ 20 $\rightarrow$ 15

4. next ()

10 $\rightarrow$ curr $\rightarrow$ 39 $\rightarrow$ 20 $\rightarrow$ 15

5. insert (12)

10 $\rightarrow$ 39 $\rightarrow$ 12 $\rightarrow$ 20 $\rightarrow$ 15 $\rightarrow$ NULL

Final current position:

curr at node 39

2. Binary Search

Array:

Index :	0	1	2	3	4	5	6	7	8
Value :	66	111	212	323	432	543	643	649	777
		9							
		888							

Algorithm:

low = 0

high = n-1

```

while (low <= high)
{
    mid = (low + high) / 2
    if (key == a[mid])
        Found
    else if (key < a[mid])
        high = mid - 1
    else
        low = mid + 1
}

```

Case 1 : Key = 111

Low	High	Mid	Value
0	9	4	432
0	3	1	111

Result : Found at Index 1.

Case 2 : Key = 888

Low	High	Mid	Value
0	9	4	432
5	9	7	649
8	9	8	777
9	9	9	888

Result : Found at index 9

Case 3 : Key = 999

low	High	Mid	Value
0	9	4	432
5	9	7	649
8	9	8	777
9	9	9	888

Now low > high.

Result : Element not found.

Time Complexity :

* Best Case : $O(1)$

* Average Case : $O(\log n)$

* Worst Case : $O(\log n)$

3. Stack (LIFO)

Array Size = 5

Operations :

push(45)

push(22)

push(11)

push(77)

Pop ()

Push (44)

push (33)

Pop ()

Pop ()

pop ()

Execution:

Operation

Stack

Push 45

45

Push 22

45 22

Push 11

45 22 11

Push 77

45 22 11 77

Pop

45 22 11

Push 44

45 22 11 44

Push 33

Stack Overflow (Full)

POP

45 22 11

Pop

45 22

Pop

45

Final Stack :

Top

↓

45

Index:

0

Peak Element = 45

Index Position = 0

Time Complexity:

* Push = $O(1)$

* Pop = $O(1)$

* Peek = $O(1)$

4. Expression Conversion

Given:

$(a+b)+c*(d+e)+f)*(g+h)$

Prefix:

* + + tab * c + de f + gh

Without Spaces:

* + + tab * c + de f + gh

Postfix:

ab+cde+*+f+gh+*

5. Queue VS Priority Queue

Simple Queue (FIFO)

- * Customers served in arrival order.
- * Fair for everyone.
- * VIP Customers cannot skip the queue.
- * Easy to implement.
- * Enqueue: $O(1)$
- * Dequeue: $O(1)$

Priority Queue

- * VIP customers are served first.
- * Reduces VIP waiting time.
- * Normal customers may wait longer.
- * Can cause starvation.
- * Heap implementation:
 - * Insert: $O(\log n)$
 - * Delete: $O(\log n)$

Comparison:

Feature	Queue	Priority Queue
Order	FIFO	Priority Based
Fairness	High	lower
VIP support	No	Yes
Waiting Time	Equal	VIP gets less waiting time
Insertion	$O(1)$	$O(\log n)$
Deletion	$O(1)$	$O(\log n)$

Conclusion:

A simple queue is best when fairness is important, while a priority queue is suitable when urgent or VIP customers need faster service, though it increases implementation complexity and may delay regular customers.